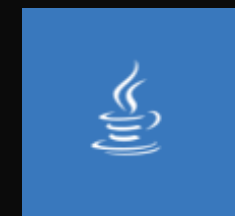


Tipos de datos y operadores

Desarrollo Orientado a Objetos



Contenido

01

¿Qué son los tipos de datos en Java?

02

Tipos de datos primitivos en Java

03

Primitivos vs Referencia

04

Operadores Aritméticos

05

Operadores de Comparación

06

Operadores Lógicos

07

Precedencia de Operadores

08

Elegir el Operador Correcto

09

Ejemplos Combinados

10

Errores Comunes

11

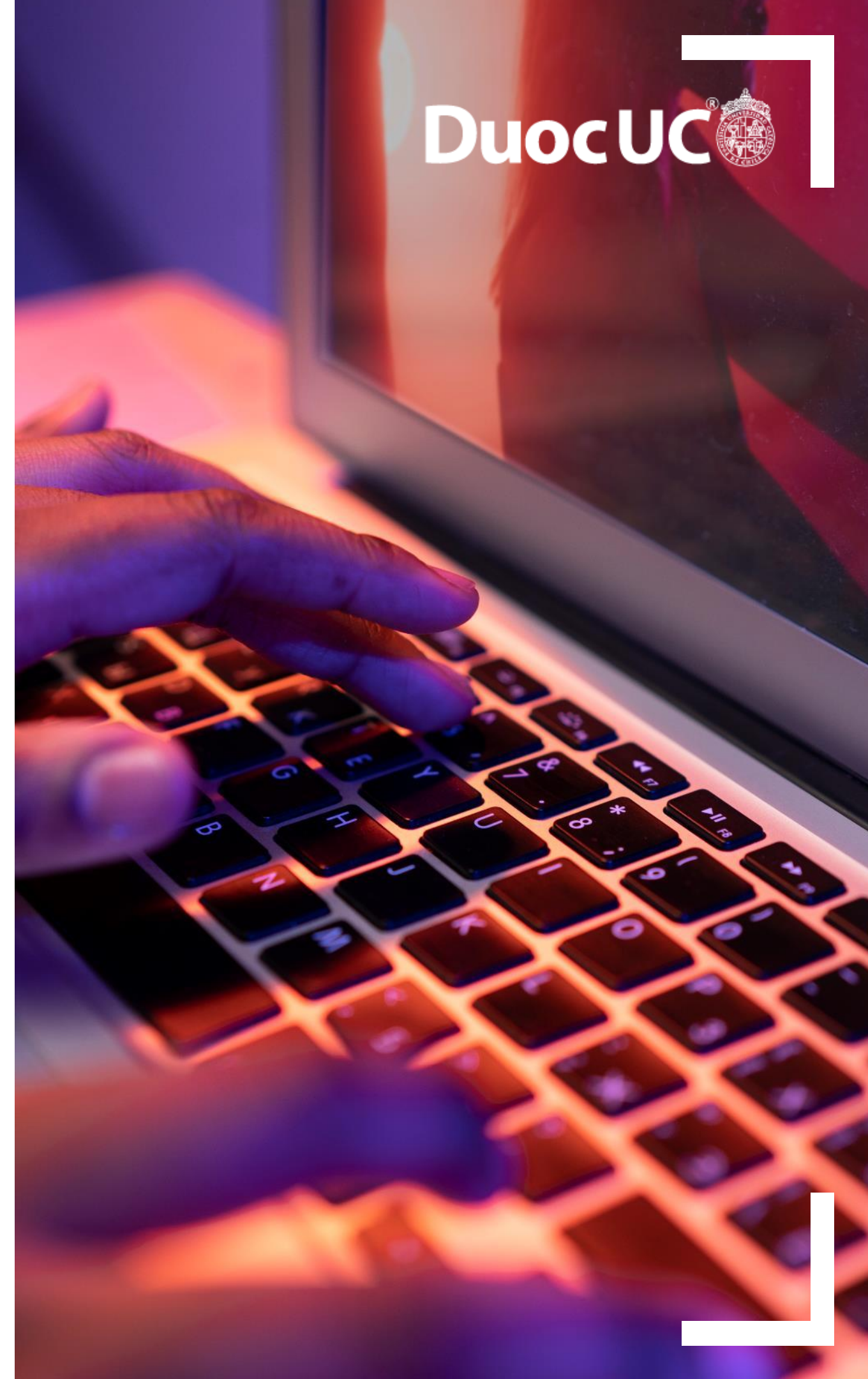
Buenas prácticas en Java

12

Tipos de Datos y Operadores

13

Conclusiones y Próximos pasos



 Fundamentos del Lenguaje

Java

Tipos de Datos y Operadores

Primitivos, referencia, operadores y precedencia.



Tipos de Datos
int, double, boolean



Operadores
+, -, ==, &&

```
● ● ● Basics.java

// 1. Tipos Primitivos y Referencia
int edad = 20;
double precio = 99.99;
boolean esEstudiante = true;
String nombre = "Ana";

// 2. Operadores y Precedencia
double total = precio * 0.8; // 20% desc

// 3. Operadores Lógicos y Comparación
if (edad ≥ 18 && esEstudiante){
    System.out.println("Descuento aplicado a " + nombre);
}
```



¿Qué son los Tipos de Datos en Java?

La base de la información en el código

Definición

En Java, cada variable debe tener un **tipo de dato** específico. Este define la naturaleza del valor que puede almacenar y cómo se comportará.



Indican el valor

Especifican si la variable guarda un número, un texto o un estado lógico.



Determinan recursos

Asignan la cantidad de memoria y definen las operaciones válidas posibles.



Evitan errores

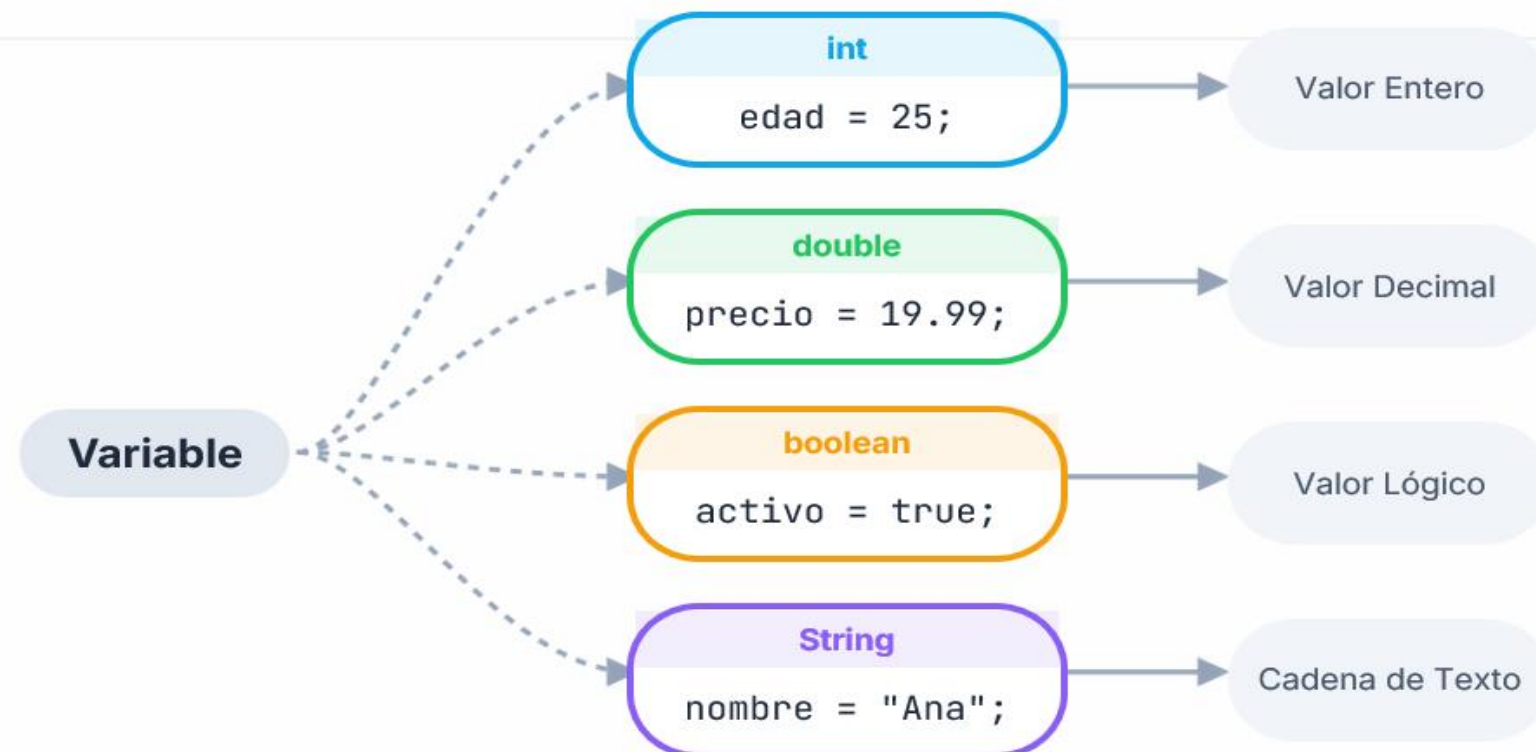
Al ser un lenguaje "fuertemente tipado", Java revisa los tipos al compilar.

Importante:



Java exige declarar el tipo antes de usar una variable.

Estructura: Variable → Tipo → Valor



Tipos de Datos Primitivos en Java

Java define ocho tipos primitivos de datos para manejar valores básicos de forma eficiente en memoria.

Enteros

byte 8-bit (-128 a 127)

short 16-bit (-32,768 a 32,767)

int 32-bit (Estándar)

long 64-bit (Sufijo 'L')

 El tipo **int** es el más utilizado para cálculos generales.

Punto Flotante

float 32-bit (Sufijo 'f')

double 64-bit (Precisión doble)

Texto

char 16-bit Unicode

Usa comillas simples para **char**: 'A'

Lógica

boolean true / false

¿Por qué Primitivos?

Mayor rendimiento (stack memory).

No son objetos (sin overhead).

Tienen valores por defecto (0, false, \u0000).





Primitivos vs Referencia

Diferencias en memoria y comportamiento



Gestión de Memoria

Java maneja los datos de dos formas distintas. La diferencia principal radica en qué es exactamente lo que se almacena en la variable.



Tipos Primitivos

Almacenan el **valor directo** en memoria. No tienen métodos y no pueden ser nulos.



Tipos de Referencia

Almacenan una **dirección** que apunta al objeto. Tienen métodos y pueden ser `null`.



Representación Visual en Memoria

Primitivos

```
int edad = 5;
```

Variable: edad

5

- ✓ Valor directo en la variable
- ✗ No tiene métodos

Referencia

```
String nombre = "Ana";
```

```
Animal p = new Animal();
```

Var: nombre

@x1A2



Objeto (Heap)

"Ana"

- ✓ Puntero a Heap
- ✓ Tiene métodos
- ! Puede ser `null`





Operadores Aritméticos

Realizando cálculos matemáticos básicos

X₁ Operadores Básicos

Se utilizan para realizar operaciones matemáticas comunes. Trabajan con tipos de datos numéricos (`int`, `double`, `float`, etc).

- +** **Suma / Resta (+, -)**
Suma o resta dos valores numéricos.
- ×** **Multiplicación / División (*, /)**
Multiplica o divide. Ojo con la división entera.
- %** **Módulo (%)**
Devuelve el resto de una división.
- ↑** **Incremento / Decremento (++ , --)**
Aumenta o disminuye el valor en 1.

Ejemplos Prácticos

+

Suma Total

```
int total = a + b;
```

/

Promedio

```
double prom = suma / cant;
```

%

Resto (Módulo)

```
int resto = n %2;
```

Calculadora.java

```
// División entera vs decimal
int divEnt = 5 / 2; // Resultado: 2
double divDec = 5.0 / 2.0; // Res: 2.5

// Incremento
int contador = 0;
contador++; // vale 1

// Módulo (útil para pares)
boolean esPar = (10 % 2) == 0; // true
```



Tipos y Operadores





Operadores de Comparación

Evaluando condiciones que devuelven verdadero o falso

= Símbolos Relacionales

Comparan dos valores y **siempre devuelven un boolean** (true o false). Son la base para tomar decisiones.

= Igual a
Verifica si dos valores son idénticos.

≠ Distinto de
Verifica si los valores son diferentes.

> Mayor que
Estrictamente mayor (no incluye igualdad).

< Menor que
Estrictamente menor.

≥ / ≤ Mayor/Menor o igual
Incluye el valor de límite.

Ejemplos Visuales



edad ≥ 18

¿Es mayor o igual a 18?

true



nota = 10

¿La nota es exactamente 10?

false



saldo ≠ 0

¿El saldo es diferente de cero?

true



¡Cuidado con la Igualdad!

En Java, = se usa para **asignar** un valor a una variable. Para **comparar** si dos valores son iguales, debes usar el doble igual ==



Operadores Lógicos

Combinando múltiples condiciones

Tipos de Operadores

Los operadores lógicos se utilizan para combinar múltiples expresiones booleanas (verdadero/falso) y tomar decisiones complejas.



AND Lógico (Y)

Verdadero SOLO si **ambas** condiciones son verdaderas.



OR Lógico (O)

Verdadero si **al menos una** condición es verdadera.



NOT Lógico (NO)

Invierte el valor: verdadero se hace falso y viceversa.

Cortocircuito:

Java evalúa de izquierda a derecha y se detiene en cuanto el resultado es seguro (ej. en `false && ...` no evalúa el resto).

Flujos de Decisión

Ejemplo: Permitir Acceso

esAdmin (`true`)

&&

activo (`true`)



✓ `true`

El acceso se concede si es administrador Y su cuenta está activa.

Ejemplo: Mostrar Contenido

tieneSaldo (`false`)

||

esInvitado (`true`)



✓ `true`

Se muestra si tiene saldo O si es un usuario invitado.

Ejemplo: Validar Estado



error (`false`)



✓ `true (es válido)`

Si NO hay error, entonces el estado es válido.



Precedencia de Operadores

El orden importa al evaluar expresiones

¿Qué es la Precedencia?

Determina el **orden en que se evalúan** los operadores en una expresión. No todos tienen la misma prioridad.



1. Prioridad Automática

Multiplicar se hace antes que sumar, igual que en matemáticas.



2. Control con Paréntesis

Los paréntesis `()` tienen la mayor prioridad y alteran el orden natural.



3. Evaluación Izq-Der

A igual prioridad, la mayoría se evalúa de izquierda a derecha.

Regla de oro: Usa paréntesis para que el código sea fácil de leer.

Ranking de Prioridad

1º Paréntesis

`( )`

2º Unarios

`++ -- !`

3º Multiplicación, División, Módulo

`* / %`

4º Suma y Resta

`+ -`

5º Comparación

`> < ≥ ≤ = ≠`

6º Lógicos (AND, OR)

`&& ||`

$2 + 3 * 4 = 14$
(Multiplica primero)

$(2 + 3) * 4 = 20$
(Suma primero)



Elegir el Operador Correcto

Guía práctica para decidir qué operador usar

? ¿Qué necesitas hacer?

Elige la categoría de operador según el objetivo de tu código.

-  **Cálculos Matemáticos:** Usa operadores aritméticos (+, -, *, /) para manipular números.
-  **Tomar Decisiones:** Usa operadores de comparación (==, >, <) para condiciones en if/else.
-  **Combinar Reglas:** Usa operadores lógicos (&&, ||) cuando hay múltiples condiciones.

Consejo:

💡 Identifica el tipo de resultado que esperas: ¿un número o un boolean?

Árbol de Decisión Visual





Ejemplos Combinados

Aplicando tipos de datos y operadores en casos reales



Casos Prácticos

En el desarrollo real, combinamos constantemente diferentes **tipos de datos** y **operadores** para resolver problemas de lógica de negocio.



Cálculos Aritméticos

Usando int y double para precios, impuestos y totales.



Lógica Booleana

Evaluando múltiples condiciones con operadores lógicos.



Cadenas (Strings)

Concatenando texto y variables para generar mensajes.

</> Expresiones en Código Java

1. Validación Compleja (boolean)

```
int nota = 75;
int asistencia = 85;
boolean aprobado = (nota >= 60) && (asistencia >= 80);
// aprobado será true
```

2. Cálculo de Precios (double)

```
double subtotal = 150.00;
double impuesto = 0.21;
double precioFinal = subtotal * (1+ impuesto);
// precioFinal será 181.5
```

3. Concatenación de Texto (String)

```
String nombre = "Ana";
String saludo = "¡Hola, "+ nombre +"! Tu saldo es $" + precioFinal;
System.out.println(saludo);
// Muestra: ¡Hola, Ana! Tu saldo es $181.5
```



Tipos de Datos



Ejemplos



⚠ Errores Comunes

Identifica y soluciona problemas frecuentes al usar tipos y operadores

🛠 ¿Qué suele salir mal?

= Confundir = con ==

Usar asignación (=) donde se requiere comparación (==).

” Comparar String con ==

Compara referencias de memoria, no el contenido del texto.

÷ División entre enteros

Dividir 5 / 2 da como resultado 2, no 2.5 (se pierde el decimal).

⚡ Olvidar paréntesis

La precedencia puede alterar el resultado de operaciones matemáticas y lógicas.

⊘ Null en referencias

Intentar usar métodos o propiedades en un objeto no inicializado (null).

🔧 Error Consecuencia Solución

Comparar Textos

`if (nombre = "Ana")` → Falla si es otro objeto String

`if (nombre.equals("Ana"))` ✓

División Inesperada

`double res = 5 / 2;` → Devuelve 2.0 (pierde decimal)

`double res = 5.0 / 2;` ✓ Devuelve 2.5

Orden de Evaluación

`int x = 2 + 3 * 4;` → `x = 14` (Multiplica primero)

`int x = (2 + 3) * 4;` ✓ `x = 20` (Suma primero)



Buenas Prácticas en Java

Consejos para escribir código limpio, seguro y eficiente

Reglas de Oro

Nombres Claros

Usa variables descriptivas (`precioFinal` en vez de `p`).

Tipo Adecuado

No uses `double` para dinero exacto (usa `BigDecimal`) ni `int` para IDs largos.

Uso de Paréntesis

Mejora la legibilidad en expresiones largas, incluso si no son estrictamente necesarios.

Validar Null

Verifica objetos antes de usar sus métodos para evitar `NullPointerException`.

Checklist Visual: Lo que SÍ debes hacer



Comparar Strings con `.equals()`

```
if (nombre.equals("Admin")) { ... }
```



Paréntesis para Legibilidad

```
boolean ok = (edad > 18) && (saldo > 0);
```



Constantes en Mayúsculas

```
final double IMPUESTO_IVA = 0.21;
```



Nombrar Booleanos como Preguntas

```
boolean esMayor = true; // no solo "mayor"
```



Tipos de Datos y Operadores

Fundamentos esenciales para la manipulación de información en programación



Tipos Primitivos

String

Number

Boolean

Null

Undefined

Son los bloques básicos de información.
Son inmutables y se pasan por valor.

```
let nombre = "Juan";  
let edad = 25;  
let esActivo = true;
```

- ✓ Almacenamiento directo en stack
- ✓ Acceso rápido y eficiente



Operadores Lógicos

Aritméticos & Comparación

&& AND Lógico

|| OR Lógico

=== Estricto

!== Diferente

Permiten realizar comparaciones y transformaciones sobre los datos.

```
if (edad > 18 && esActivo) {  
  console.log("Acceso ok");  
}
```



Objetos & Memoria

Object

Array

Function

Estructuras complejas que se pasan por referencia. Almacenadas en el Heap.

```
const user = {  
  id: 1,  
  roles: ['admin']  
};
```

"Modificar una propiedad en un objeto copiado afecta al original si no se realiza una copia profunda."



Conclusiones y Próximos Pasos

Cierre de Tipos y Operadores

Ideas Clave

Comprender los **tipos de datos** y **operadores** es fundamental para crear expresiones correctas y evitar errores sutiles en Java.



El tipo define el dato

Elige el tipo adecuado según la necesidad de memoria y precisión.



El operador define la acción

Usa el operador correcto para cálculos, comparaciones o lógica.



Precedencia y Claridad

El orden importa. Usa paréntesis para asegurar el resultado y la legibilidad.

Resultado:

• **Código más correcto, legible y seguro**

Estructura Fundamental



→ Próximos Pasos

1 Condicionales (if/else)

2 Casting de Tipos

3 Expresiones Complejas



Felicitaciones
¡Has finalizado el módulo!

DuocUC[®]



CERCANÍA. LIDERAZGO. FUTURO.

7
AÑOS
ACREDITADO

 Comisión Nacional
de Acreditación
CNA-Chile

NIVEL DE EXCELENCIA
HASTA OCTUBRE 2031

Docencia de pregrado / Gestión institucional / Aseguramiento interno de la
calidad / Vinculación con el Medio / Investigación, creación y/o innovación